

CSR Automation Toolkit — Setup Guide

Generic prototype — works with no setup; Slack, Sheets, and OpenAI are optional add-ons

No .env file needed. Earlier versions of this guide described manually editing a .env file. That step is gone — the app now has its own Setup screen, and works immediately with zero configuration (keyword matching + local CSV logging). Everything below marked "optional" can be skipped.

Prerequisites

- Python 3.8+ (only needed to run from source or build the .exe — not needed for teammates running a pre-built .exe)
- Optional: Slack workspace with admin access, for real-time query alerts
- Optional: Google Sheets + a Google Cloud service account, for team-wide audit logging
- Optional: an OpenAI API key, for AI-assisted classification of ambiguous queries

1. Running from Source

Install dependencies

```
pip install -r requirements.txt
```

Only requests is required. gspread, oauth2client, and openai are only used if you configure those optional features in Settings.

Run the app

```
python csr_automation_toolkit.py
```

On first launch, a Setup screen appears. Click **"Skip — use offline mode"** to start using the toolkit immediately with no configuration, or fill in any of the optional sections below and click **"Save and Continue."**

2. Optional: Slack Alerts

- Go to api.slack.com/apps → create an app (or use an existing one)
- Enable Incoming Webhooks, add a webhook to your workspace/channel
- Copy the Webhook URL into the "Slack Webhook URL" field in Settings

3. Optional: Google Sheets Logging

The CSV audit log always runs regardless of this setting — Sheets adds team-wide visibility on top of it, it doesn't replace it.

- Create a Google Sheet with columns: Timestamp | Customer Email | Query | Response | Follow-Up
- In Google Cloud Console, create a project and enable the Google Sheets API and Google Drive API
- Create a Service Account, generate a JSON key
- Share your Google Sheet with the service account's email address (Editor access)
- In Settings, paste the Sheet ID (from the sheet's URL) and the JSON key contents (as one line) into the corresponding fields

4. Optional: AI-Assisted Classification

- Get an API key from platform.openai.com/api-keys
- Paste it into the "OpenAI API Key" field in Settings
- If left blank, the toolkit uses keyword matching only — this is the default and requires no setup

5. Where Settings Are Stored

```
Windows: %APPDATA%\CSRToolkit\config.json
Mac: ~/Library/Application Support/CSRToolkit/config.json
Linux: ~/.config/CSRToolkit/config.json
```

Each person's credentials are stored locally on their own machine only — never bundled into a packaged .exe, never shared between teammates. The CSV audit log lives in the same folder, as `csr_logs.csv`.

6. Building a Standalone .exe

For distributing to non-technical teammates with no Python install required, see **PACKAGING.md** in this repository for the full PyInstaller build steps.

7. Troubleshooting

Issue	What to check
"Failed to send Slack alert"	Confirm the Webhook URL in Settings is correct and the workspace/channel still exists.
"Failed to log to Sheets"	Confirm the service account email is shared on the Sheet (Editor access), and the Sheet ID and JSON are both correct.
"AI classification error"	Confirm the OpenAI key is valid and has available credits. Falls back to keyword matching either way.
GUI doesn't open	Confirm tkinter is installed (included with most Python installs; on Mac: brew install python-tk@3.11)

8. Next Steps

- Expand the response categories beyond the current proof-of-concept set
- Pair with the sanitized CSR query library for a fuller, brand-agnostic category set
- Package as a .exe and pilot with 1–2 teammates before wider rollout
- Review the CSV/Sheets audit log regularly and refine responses based on real usage